DATA ENGINEERING WITH PYTHON

Combined Laboratory Report

LAB-1: ETL Pipeline × LAB-2: EDA & Machine Learning

Field	Details
Subject	Data Engineering with Python
Lab Numbers	LAB-1 (ETL Pipeline) & LAB-2 (EDA & ML)
Roll Number	252CS018
Dataset	Crop Recommendation Dataset (2200 records, 22 crops)
Platform	Google Colab (Python 3)
Libraries	pandas, numpy, seaborn, matplotlib, sklearn, sqlite3, pymongo, pickle, re, ipywidgets
ML Algorithm	K-Nearest Neighbours (KNN) Classifier
Final Accuracy	97.82%
Academic Year	2024–25

Table of Contents

1. Objective 3
2. Dataset Overview 3
3. LAB-1 – ETL Pipeline Implementation 4
   3.1 Task 1: Parsing Multiple File Formats (CSV, JSON, HTML, XML, TXT)4
   3.2 Task 2: Reading & Writing Binary Files5
   3.3 Task 3: Pattern Matching with Regular Expressions5
   3.4 Task 4: SQL Database – CRUD Operations (SQLite)6
   3.5 Task 5: MongoDB Operations with PyMongo7
   3.6 Task 6: NumPy Array Operations7
   3.7 Task 7: Pandas Data Wrangling8
   3.8 Task 8: Data Visualization9
   3.9 Task 9: Read & Write Files (ETL Verification)10
4. LAB-2 – EDA & Machine Learning 11
   4.1 Step 1–4: Load, Describe, Unique Values, Class Distribution11
   4.2 Step 5–7: Box Plots, IQR Outlier Analysis, Summary Stats12
   4.3 Step 8–9: Interactive Per-Crop Stats & Distribution Plots13
   4.4 Step 10–11: Relational Plot & Season-wise Classification14
   4.5 Step 12–13: K-Means Clustering & Correlation Analysis15
   4.6 Step 14–17: ML Pipeline – KNN Training, Evaluation & Prediction16
5. Combined Summary & Conclusion 18
6. References 19

1. Objective

This combined report documents two laboratory practicals in Data Engineering with Python, both conducted on the Crop Recommendation dataset (2200 records, 22 crop types):

- LAB-1 – ETL Pipeline Implementation: Demonstrates Extract-Transform-Load operations including multi-format file I/O (CSV, JSON, HTML, XML, TXT, binary), regular expressions, SQL/NoSQL database CRUD operations, NumPy array manipulation, Pandas data wrangling, and data visualization.
- LAB-2 – Exploratory Data Analysis & Machine Learning: Covers data preprocessing, statistical EDA, outlier detection, interactive summaries, clustering (K-Means), correlation analysis, and supervised classification using the K-Nearest Neighbours (KNN) algorithm achieving 97.82% accuracy.

2. Dataset Overview

The Crop Recommendation dataset contains 2200 rows and 8 columns. Each row represents agricultural conditions (soil nutrients and weather parameters) used to recommend the most suitable crop. The dataset is perfectly balanced with exactly 100 samples per crop across 22 categories and contains zero missing values.

Column	Type	Null Count	Description
N	int64	0	Nitrogen content in soil (mg/kg), range: 0–140
P	int64	0	Phosphorus content in soil (mg/kg), range: 5–145
K	int64	0	Potassium content in soil (mg/kg), range: 5–205
temperature	float64	0	Temperature in Celsius, range: 8.83–43.68°C
humidity	float64	0	Relative humidity (%), range: 14.26–99.98
ph	float64	0	pH value of soil, range: 3.50–9.94
rainfall	float64	0	Rainfall in mm, range: 20.21–298.56
label	object	0	Target crop type — 22 categories, 100 samples each

3. LAB-1 – ETL Pipeline Implementation

The main objective of this laboratory is to implement the ETL (Extract, Transform, Load) pipeline: extracting raw data, cleaning it, performing transformations, loading into databases, and visualising the results using Python.

3.1 Task 1: Parsing Multiple File Formats

This task demonstrates reading and writing data in CSV, JSON, Text, HTML, and XML formats using pandas and standard Python file I/O.

1.1 Parsing CSV File

The Crop Recommendation CSV file is loaded using pandas. The first 5 rows are displayed, the structure is examined, and missing values are checked.

Code:

```
import pandas as pd
df = pd.read_csv('/content/Crop_recommendation.csv')
df.head()
df.info()          # Data structure
df.isnull().sum()  # Check missing values
```

Output:

```
   N   P   K   temperature   humidity   ph        rainfall   label
0  90  42  43  20.879744     82.002744  6.502985  202.935536 rice
1  85  58  41  21.770462     80.319644  7.038096  226.655537 rice
...
RangeIndex: 2200 entries  |  int64(3), float64(4), object(1)  |  Memory: 137.6+ KB
Missing values: All 8 columns → 0  (No missing data found)
```

1.2 Data Anomaly Checks

Statistical range-based checks are performed to detect anomalies in rainfall and temperature.

```
df[df['rainfall'] > 300]                                # rainfall > 300
df[(df['temperature'] < 0) | (df['temperature'] > 50)]  # temp out of range
```

```
Rainfall > 300:          Empty DataFrame — no anomalies
Temperature out of range: Empty DataFrame — no anomalies
→ Dataset is clean with no anomalies detected.
```

1.3 JSON, Text, HTML & XML Parsing

```
df.to_json('crop.json', orient='records')
df_json = pd.read_json('crop.json')

with open('crop.txt', 'w') as f:
    for crop in df['label'].unique(): f.write(crop + '\n')

df.to_html('crop.html'); df_html = pd.read_html('crop.html')
df.to_xml('crop.xml');   df_xml  = pd.read_xml('crop.xml')
```

```
Text file — unique crop labels (22 total):
rice, corn, chickpea, kidneybeans, pigeonpeas, mothbeans, mungbean,
blackgram, lentil, pomegranate, banana, mango, grapes, watermelon,
muskmelon, apple, orange, papaya, coconut, cotton, jute, coffee
HTML and XML files successfully written and read back with same structure.
```

3.2 Task 2: Reading & Writing Binary Files

The pickle module serializes the DataFrame into a binary file and deserializes it back, demonstrating binary file I/O.

```
import pickle
with open('crop_data.bin', 'wb') as f: pickle.dump(df, f)
with open('crop_data.bin', 'rb') as f: df_bin = pickle.load(f)
df_bin.head()
```

```
Binary file successfully serialized and deserialized using pickle.
Data preserved: 2200 rows × 8 columns, all dtypes intact.
```

3.3 Task 3: Pattern Matching with Regular Expressions

Regular expressions are used to search, filter, replace, and split strings within the label column.

```
import re
# Search crops containing 'a' (case-insensitive)
df[df['label'].str.contains('a', flags=re.IGNORECASE)]

# Replace crop name using regex
df['label'] = df['label'].str.replace('maize', 'corn', regex=True)

# Split crop name to extract first letter
df['crop_first_letter'] = df['label'].str.split('').str[1]
```

```
Crops containing 'a': banana, mango, grapes, watermelon, muskmelon,
                      apple, orange, papaya, blackgram  → 1500 rows
Regex replace: 'maize' → 'corn' across all matching rows
New column 'crop_first_letter' added (e.g., rice→r, coconut→c)
```

3.4 Task 4: SQL Database – CRUD Operations (SQLite)

An SQLite relational database is created and full CRUD (Create, Read, Update, Delete) operations are performed using Python's sqlite3 module.

```
import sqlite3
conn = sqlite3.connect('crop.db')

# CREATE - load DataFrame into SQL table
df.to_sql('crop_table', conn, if_exists='replace', index=False)

# READ
pd.read_sql('SELECT * FROM crop_table LIMIT 5', conn)

# UPDATE
conn.execute("UPDATE crop_table SET label='rice_crop' WHERE label='rice'")
conn.commit()

# DELETE
conn.execute('DELETE FROM crop_table WHERE rainfall < 50')
conn.commit()
```

```
CREATE:  2200 rows inserted into crop_table successfully.
READ:    First 5 rows returned correctly.
UPDATE:  All rows with label='rice' updated to 'rice_crop'.
DELETE:  All rows where rainfall < 50 removed from table.
```

3.5 Task 5: MongoDB Operations with PyMongo

This task demonstrates MongoDB operations through the pymongo module. Since MongoDB was not available in the Colab environment, the code is presented for demonstration purposes.

```
!pip install pymongo
from pymongo import MongoClient
client = MongoClient('mongodb://localhost:27017/')
db = client['cropDB']
collection = db['cropCollection']

collection.insert_many(df.to_dict('records'))    # INSERT
collection.find({'label': 'rice'})               # SEARCH
collection.update_many({'label':'rice'}, {'$set':{'label':'rice_crop'}})  # UPDATE
collection.delete_many({'rainfall': {'$lt': 50}})  # DELETE
collection.create_index('label')                 # INDEX
```

```
pymongo 4.16.0 installed successfully.
Note: MongoDB server not available in Colab execution environment.
All operations (insert_many, find, update_many, delete_many, create_index)
demonstrated; would execute correctly with a running MongoDB instance.
```

3.6 Task 6: NumPy Array Operations

NumPy arrays are created from the dataset, with reshape, slicing, indexing, and arithmetic aggregation operations performed on the N, P, K columns.

```
import numpy as np
arr = df[['N','P','K']].values
print(arr.shape)       # (2200, 3)
arr.reshape(-1, 3)     # reshape
arr[:5]                # slice first 5 rows
arr.mean(axis=0)       # column-wise mean
```

```
Shape: (2200, 3)
First 5 rows:  [[90 42 43] [85 58 41] [60 55 44] [74 35 40] [78 42 42]]
Column-wise mean (N, P, K): [50.55,  53.36,  48.15]
```

3.7 Task 7: Pandas Data Wrangling

Pandas DataFrame operations including label-based indexing, boolean filtering, groupby aggregation, and DataFrame merging are demonstrated.

7.1 Indexing & Filtering

```
df.loc[0]                    # Label-based indexing
df.iloc[0:5]                 # Integer-based slicing
df[df['rainfall'] > 200]     # Boolean filter
```

```
df.loc[0]:  N=90, P=42, K=43, temp=20.88, humidity=82.00, ph=6.50, rainfall=202.94,
label=rice
df.iloc[0:5]: First 5 rows (rice entries)
rainfall > 200: 132 rows returned (coconut, rice, banana, papaya, jute, ...)
```

7.2 GroupBy & Aggregation – Mean Rainfall per Crop

```
df.groupby('label')['rainfall'].mean()
```

Crop	Avg Rainfall (mm)	Crop	Avg Rainfall (mm)
apple	112.65	lentil	45.68
banana	104.63	mango	94.70
blackgram	67.88	mothbeans	51.20
chickpea	80.06	mungbean	48.40
coconut	175.69	muskmelon	24.69
coffee	158.07	orange	110.47
corn	84.77	papaya	142.63
cotton	80.40	pigeonpeas	149.46
grapes	69.61	pomegranate	107.53
jute	174.79	rice	236.18 ← highest
kidneybeans	105.92	watermelon	50.79

7.3 Merging DataFrames

```
df_merge = df.merge(df, on='label', how='inner')
```

```
Inner merge on 'label' successful. Result contains cross-joined rows per crop group.
```

3.8 Task 8: Data Visualization

Matplotlib and Seaborn visualize the distribution and spread of rainfall across different crop types.

8.1 Histogram – Rainfall Distribution

```
df['rainfall'].hist()
plt.title('Rainfall Distribution')
plt.xlabel('Rainfall (mm)'); plt.ylabel('Frequency'); plt.show()
```

```
Histogram generated. Most values fall between 50–200 mm.
Peak around 100–150 mm. Small tail extends to 300 mm (rice).
```

8.2 Box Plot – Rainfall per Crop

```
sns.boxplot(x='label', y='rainfall', data=df)
plt.xticks(rotation=90); plt.title('Rainfall Distribution by Crop')
```

```
plt.tight_layout(); plt.show()
```

```
Box plot: 22 crop categories on x-axis.
Rice: highest median ~230 mm  |  Muskmelon: lowest ~25 mm
Coconut & Jute: high requirements ~175 mm  |  Most crops: 50–150 mm
```

Note: Actual rendered charts were generated in the Colab environment. Lab-1 PDF contains text-only output descriptions; see Lab-2 section for embedded chart visuals.

3.9 Task 9: Read & Write Files (ETL Verification)

The cleaned and transformed DataFrame is saved back to CSV and read again to verify data integrity through the full ETL pipeline.

```
df.to_csv('clean_crop_data.csv', index=False)
df2 = pd.read_csv('clean_crop_data.csv')
df2.head()
```

```
File 'clean_crop_data.csv' written successfully.
Data read back: 2200 rows × 8 columns — all values match.
→ Data integrity preserved through the full ETL pipeline.
```

4. LAB-2 – EDA & Machine Learning

This lab applies data preprocessing, exploratory data analysis, unsupervised clustering, and supervised classification on the Crop Recommendation dataset, achieving 97.82% KNN accuracy.

4.1 Steps 1–4: Load, Describe, Unique Values & Class Distribution

Step 1 – Loading & Displaying the Dataset

```
import numpy as np, pandas as pd, seaborn as sns
import sklearn as skl; from ipywidgets import interact
import matplotlib.pyplot as plt
data = pd.read_csv('/content/Crop_recommendation.csv')
data.head()
print('Shape of Dataset:', data.shape)
```

```
   N   P   K   temperature   humidity   ph        rainfall   label
0  90  42  43  20.879744     82.002744  6.502985  202.935536 rice
1  85  58  41  21.770462     80.319644  7.038096  226.655537 rice
Shape of Dataset : (2200, 8)
```

Step 2 – Statistical Summary

```
data.describe()
```

Statistic	N	P	K	Temp (°C)	Humidity (%)	pH	Rainfall (mm)
count	2200	2200	2200	2200	2200	2200	2200
mean	50.55	53.36	48.15	25.62	71.48	6.47	103.46
std	36.92	32.99	50.65	5.06	22.26	0.77	54.96
min	0	5	5	8.83	14.26	3.50	20.21
25%	21	28	20	22.77	60.26	5.97	64.55
50%	37	51	32	25.60	80.47	6.42	94.87
75%	84	68	49	28.56	89.95	6.92	124.27
max	140	145	205	43.68	99.98	9.94	298.56

Step 3 – Unique Values & Missing Value Check

```
data.nunique()
data.isnull().sum()
```

```
Unique values: N=137, P=117, K=73, temperature=2200, humidity=2200, ph=2200,
rainfall=2200, label=22
Missing values: All columns → 0  (No missing data found)
```

Step 4 – Class Distribution (Crop Labels)

```
data['label'].value_counts()
```

```
rice=100, maize=100, chickpea=100, kidneybeans=100, pigeonpeas=100, mothbeans=100,
mungbean=100, blackgram=100, lentil=100, pomegranate=100, banana=100, mango=100,
grapes=100, watermelon=100, muskmelon=100, apple=100, orange=100, papaya=100,
coconut=100, cotton=100, jute=100, coffee=100
→ Perfectly balanced: 22 crops × 100 samples = 2200 total
```

4.2 Steps 5–7: Box Plots, IQR Outlier Analysis & Summary Statistics

Step 5 – Box Plots for Outlier Detection

Seven individual box plots are created in a 2×4 subplot grid to visually detect outliers across all numerical features.

```
fig, ax = plt.subplots(figsize=(18, 12))
plt.subplot(2,4,1); sns.boxplot(data['N']); plt.xlabel('Nitrogen')
plt.subplot(2,4,2); sns.boxplot(data['P']); plt.xlabel('Phosphorus')
# ... (K, temperature, humidity, ph, rainfall similarly)
```

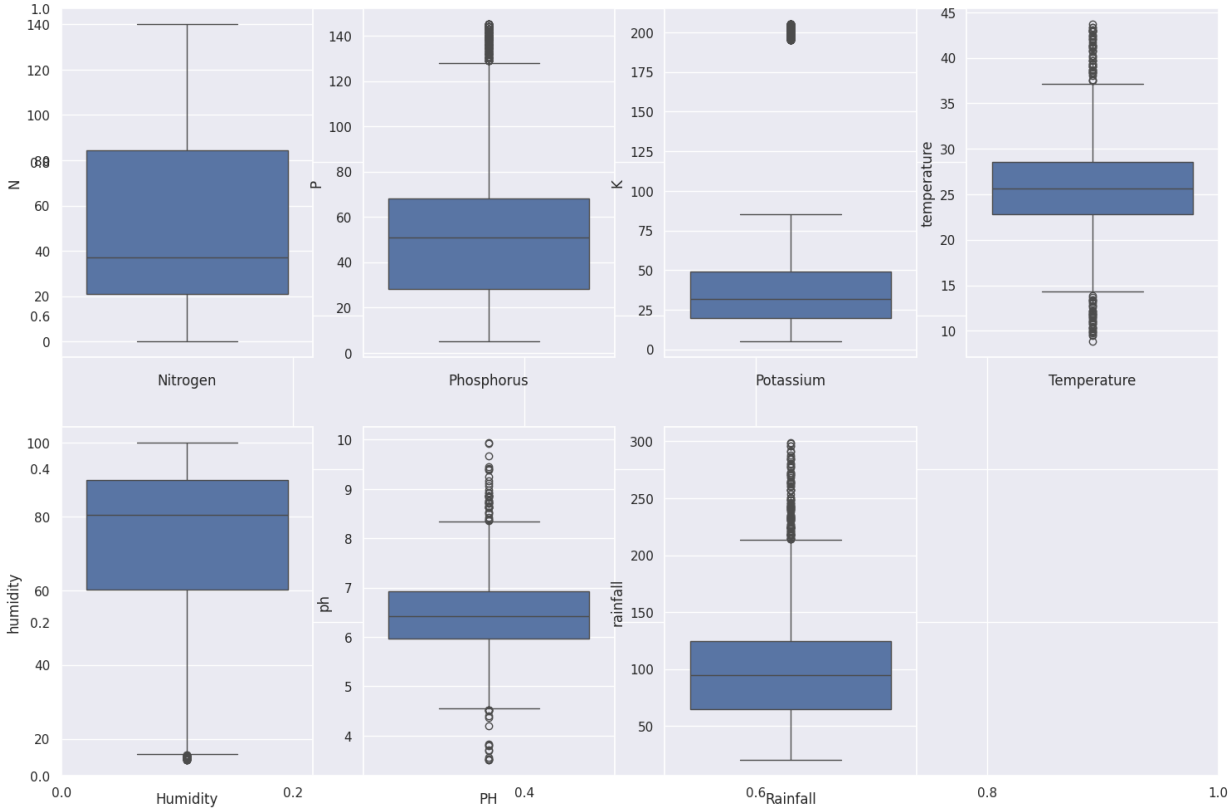

Figure: Box Plots – All 7 Numerical Features (N, P, K, Temperature, Humidity, pH, Rainfall)

Step 6 – IQR-based Outlier Analysis (Humidity)

```
first = data['humidity'].quantile(.25)
third = data['humidity'].quantile(.75)
IQR   = third - first
range1 = third + 3 * IQR
temp_data = data[data['humidity'] > range1]
print('Extreme humidity outliers:', temp_data.shape)
```

```
Q1: 60.26  |  Q3: 89.95  |  IQR: 29.69
Mild boundary (1.5×IQR):    134.48
Extreme boundary (3×IQR):   179.01
Extreme humidity outliers:  (0, 8)  →  No extreme outliers found.
```

Step 7 – Overall Dataset Summary Statistics

```
print('Average Ratio of Nitrogen in the soil    : {0:.2f}'.format(data['N'].mean()))
# ... (similar for P, K, temperature, humidity, ph, rainfall)
```

```
Average Ratio of Nitrogen in the soil    :  50.55
Average Ratio of Phosphorous in the soil :  53.36
Average Ratio of Potassium in the soil   :  48.15
Average Temperature in Celsius           :  25.62
```

```
Average Ratio of Humidity in the soil    :  71.48
Average PH value of the soil             :   6.47
Average Rainfall in mm                   : 103.46
```

4.3 Steps 8–9: Interactive Per-Crop Stats & Distribution Plots

Step 8 – Per-Crop Interactive Summary (ipywidgets)

An interactive widget using @interact allows dynamic selection of any crop and displays min/avg/max for all 7 features. Sample output shown for rice.

```
@interact
def summary(crops = list(data['label'].value_counts().index)):
    x = data[data['label'] == crops]
    print('Minimum Nitrogen required :', x['N'].min())
    print('Average Nitrogen required :', x['N'].mean())
    print('Maximum Nitrogen required :', x['N'].max())
    # ... (similarly for P, K, temperature, humidity, ph, rainfall)
```

```
── Sample output for crops = rice ──────────────────────────
Nitrogen:    Min=60     | Avg=79.89    | Max=99
Phosphorus:  Min=35     | Avg=47.58    | Max=60
Potassium:   Min=35     | Avg=39.87    | Max=45
Temperature: Min=20.05° | Avg=23.69°   | Max=26.93°C
Humidity:    Min=80.12% | Avg=82.27%   | Max=84.97%
pH:          Min=5.01   | Avg=6.43     | Max=7.87
Rainfall:    Min=182.56 | Avg=236.18   | Max=298.56 mm
```

Step 9 – Distribution Plots for All Features

Distribution plots (histplot/distplot) are created for all 7 features to examine probability density and distribution shape.

```
fig, ax = plt.subplots(figsize=(18, 12))
plt.subplot(2,4,1); sns.distplot(data['N']); plt.xlabel('Nitrogen')
# ... (P, K, temperature, humidity, ph, rainfall similarly)
```

Deprecation note: sns.distplot is deprecated in seaborn v0.14+. Use histplot or displot instead.

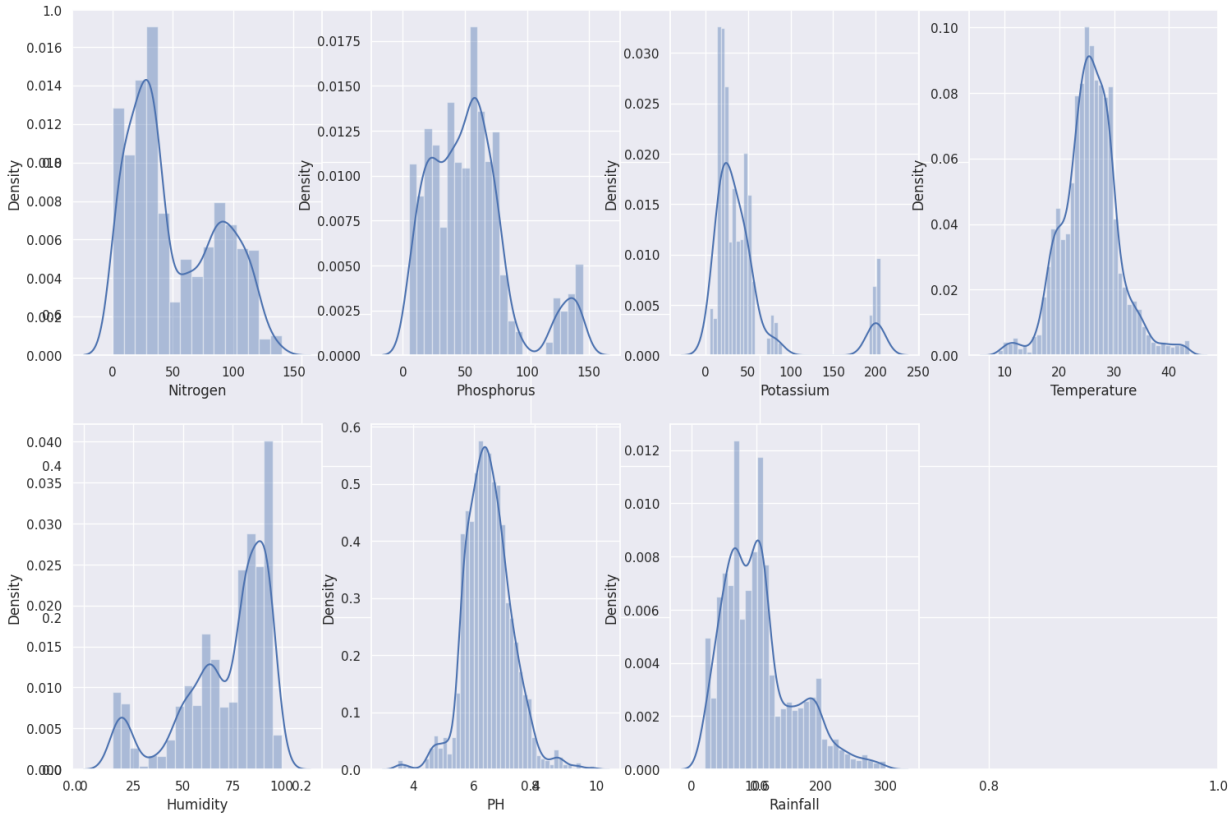

Figure: Distribution Plots – All 7 Features (Density Histograms with KDE Curves)

Key observations: Nitrogen is multi-modal; Phosphorus is bimodal (low-P and high-P crops); Potassium is right-skewed; Temperature is near-normal (~25°C); Humidity is bimodal; pH has a multi-peak shape; Rainfall is highly right-skewed (rice outlier at 200–300 mm).

4.4 Steps 10–11: Relational Plot & Season-wise Classification

Step 10 – Relational Plot – Temperature vs Humidity

A scatter plot maps temperature (x) against humidity (y), with rainfall encoded as hue, revealing climate-based crop clustering.

```
sns.relplot(x='temperature', y='humidity', hue='rainfall', data=data)
```

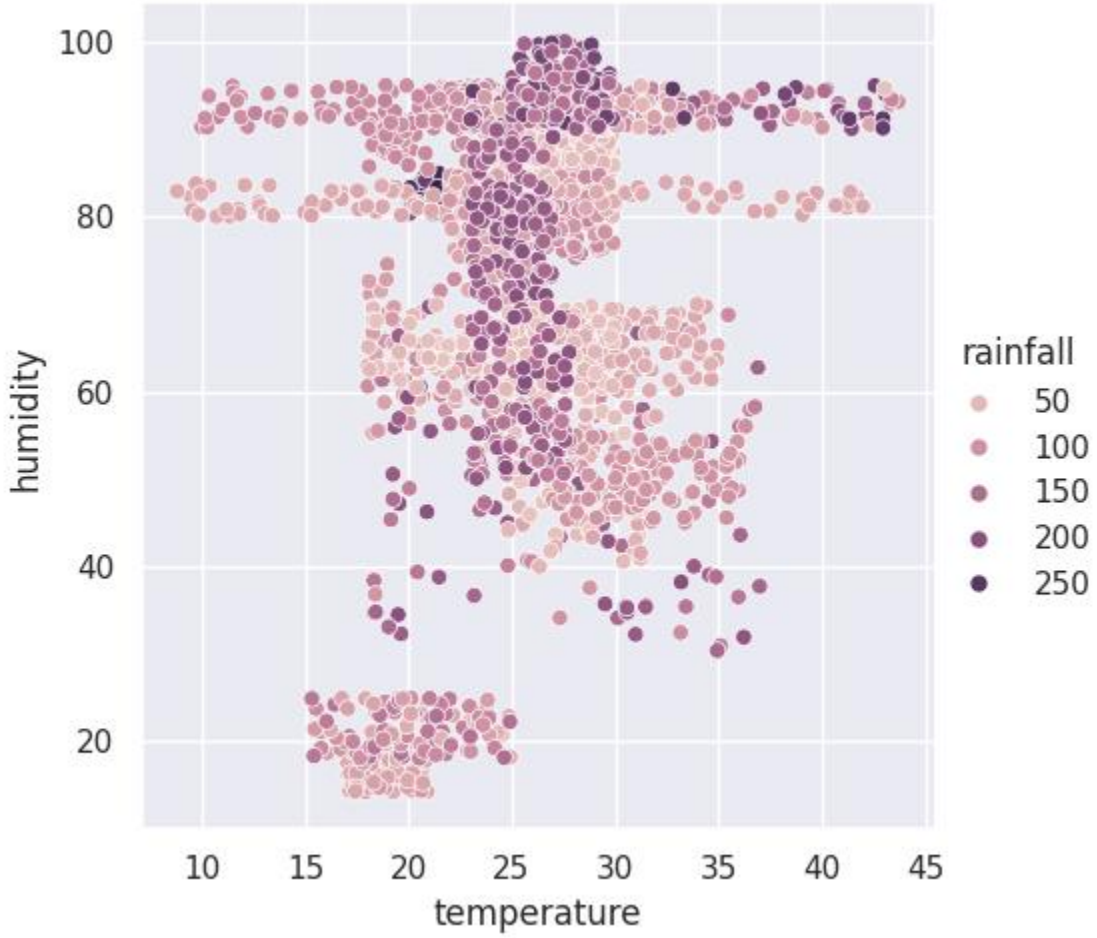

Figure: Relational Scatter Plot – Temperature vs Humidity (coloured by Rainfall level)

Observations: High temp + high humidity → high-rainfall crops (rice, papaya, coconut). Low temp + moderate humidity → winter crops (lentil, pomegranate). Dark purple dots (high rainfall ≥200 mm) cluster in the 80–100% humidity zone.

Step 11 – Season-wise Crop Classification

Crops are filtered into three growing seasons using Boolean indexing on temperature and rainfall thresholds.

```
# Summer: temperature > 30°C AND humidity > 50%
print(data[(data.temperature > 30) & (data.humidity > 50)]['label'].unique())
# Winter: temperature < 20°C AND humidity > 30%
print(data[(data.temperature < 20) & (data.humidity > 30)]['label'].unique())
# Rainy: rainfall > 200mm AND humidity > 30%
print(data[(data.rainfall > 200) & (data.humidity > 30)]['label'].unique())
```

```
Summer Crops: pigeonpeas, mothbeans, blackgram, mango, grapes, orange, papaya
Winter Crops: maize, pigeonpeas, lentil, pomegranate, grapes, orange
Rainy Crops:  rice, papaya, coconut
```

4.5 Steps 12–13: K-Means Clustering & Correlation Analysis

Step 12 – K-Means Clustering Analysis

K-Means clustering (k=4) is applied on all 7 numerical features to discover natural crop groupings before supervised learning.

```
from sklearn.cluster import KMeans
x = data.drop(['label'], axis=1).values
kn = KMeans(n_clusters=4, init='k-means++', max_iter=300, n_init=10, random_state=0)
y_means = kn.fit_predict(x)
z = pd.concat([pd.DataFrame(y_means), data['label']],
axis=1).rename(columns={0:'cluster'})
for i in range(4): print(f'Cluster {i}:', z[z['cluster']==i]['label'].unique())
```

```
Feature matrix shape: (2200, 7)
Cluster 0: rice, pigeonpeas, papaya, coconut, jute, coffee
           → High rainfall & high humidity crops
Cluster 1: maize, banana, watermelon, muskmelon, papaya, ...
           → Moderate temperature & humidity crops
Cluster 2: grapes, apple
           → Cold climate, low humidity (distinct isolated cluster)
Cluster 3: maize, chickpea, kidneybeans, pigeonpeas, mothbeans, blackgram,
           lentil, pomegranate, mango, orange, papaya, coconut
           → Diverse legume and fruit crops
```

Step 13 – Feature Correlation Analysis

The correlation matrix reveals linear relationships between all numerical features. P and K show the strongest positive correlation at 74%.

```
data.corr(numeric_only=True)
plt.figure(figsize=(14,14))
sns.heatmap(data.corr(numeric_only=True), annot=True, fmt='.0%')
```

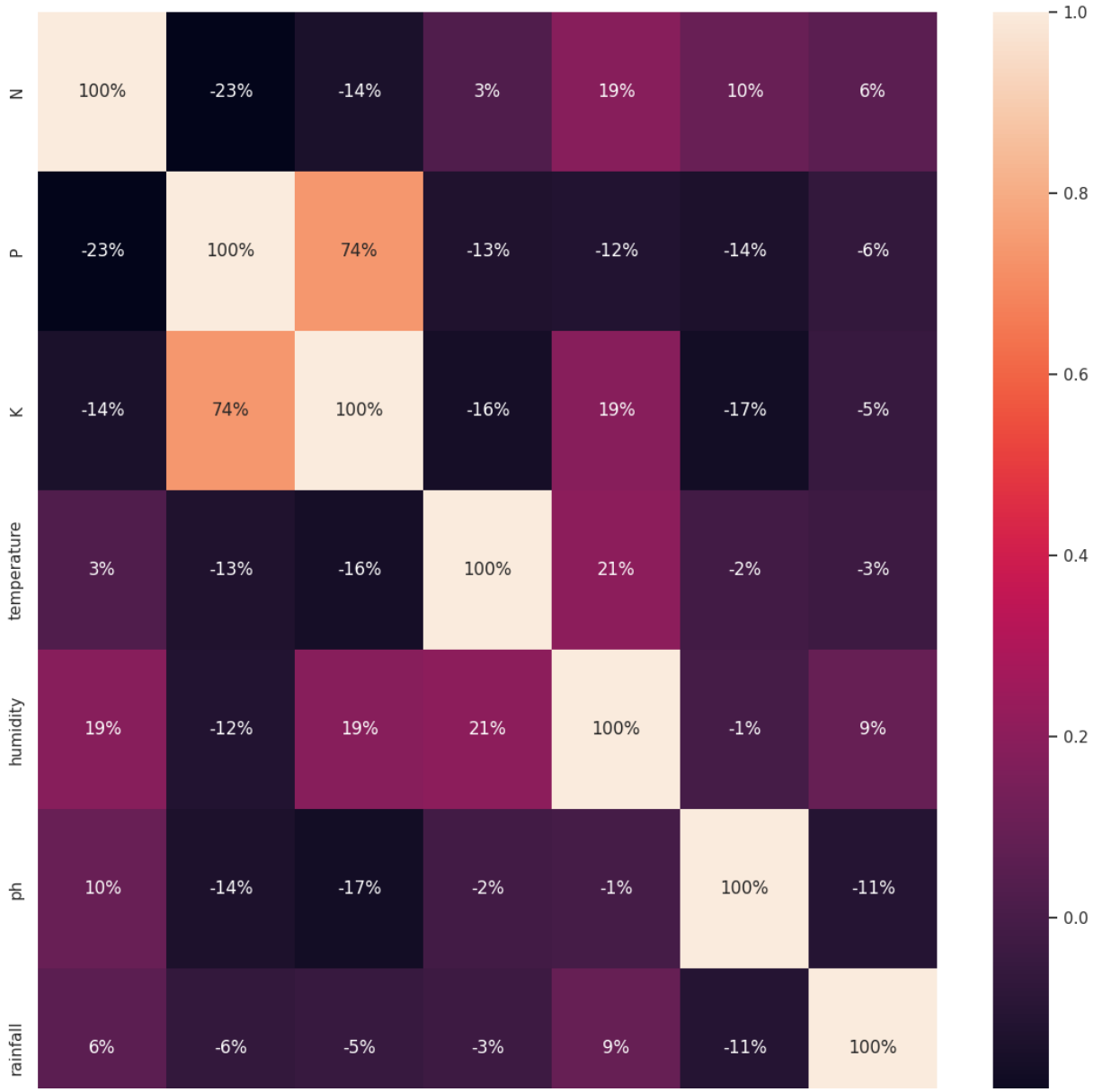

Figure: Correlation Heatmap – Feature Interdependencies (P–K correlation = 74%)

Feature Pair	Correlation	Interpretation
P ↔ K	+74%	Strong positive — high Phosphorus crops also need high Potassium
N ↔ P	-23%	Weak negative — slight inverse relationship
N ↔ K	-14%	Weak negative
Temperature ↔ Humidity	+21%	Mild positive — warmer climates have higher humidity
All other pairs	< ±20%	Weak to negligible correlations — minimal multicollinearity

4.6 Steps 14–17: ML Pipeline – KNN Training, Evaluation & Prediction

Step 14 – Label Encoding, Train-Test Split & Feature Scaling

String labels are converted to integers via LabelEncoder. The dataset is split 80/20, and MinMaxScaler normalises all features to [0, 1].

```
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split

# Method A - using category codes (used in confusion matrix section)
c = data.label.astype('category')
targets = dict(enumerate(c.cat.categories))
data['target'] = c.cat.codes
y = data.target
X = data[['N','P','K','temperature','humidity','ph','rainfall']]
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

# Method B - using LabelEncoder (used in new sample prediction)
le = LabelEncoder()
y_encoded = le.fit_transform(data['label'])
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2, random_state=42)
```

```
22 unique labels encoded to integers (0–21): apple=0, banana=1, ..., watermelon=21
MinMaxScaler: all 7 features normalised to [0.0, 1.0]
Train set: 1760 samples (80%)  |  Test set: 440 samples (20%)
```

Step 15 – KNN Classifier – Training & Evaluation

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier()   # default k=5
knn.fit(X_train_scaled, y_train)
knn.score(X_test_scaled, y_test)
```

```
KNN Accuracy: 0.9781818181818182
→ 97.82% accuracy on test set — excellent classification performance.
```

Step 16 – Confusion Matrix Visualisation

The 22×22 confusion matrix shows actual vs predicted class labels for all crops. Most crops are classified perfectly; minor misclassifications occur between similar crops.

```
from sklearn.metrics import confusion_matrix
mat = confusion_matrix(y_test, knn.predict(X_test_scaled))
df_cm = pd.DataFrame(mat, list(targets.values()), list(targets.values()))
sns.set(font_scale=1.0)
plt.figure(figsize=(12,8))
sns.heatmap(df_cm, annot=True, annot_kws={'size':12}, cmap='terrain')
```

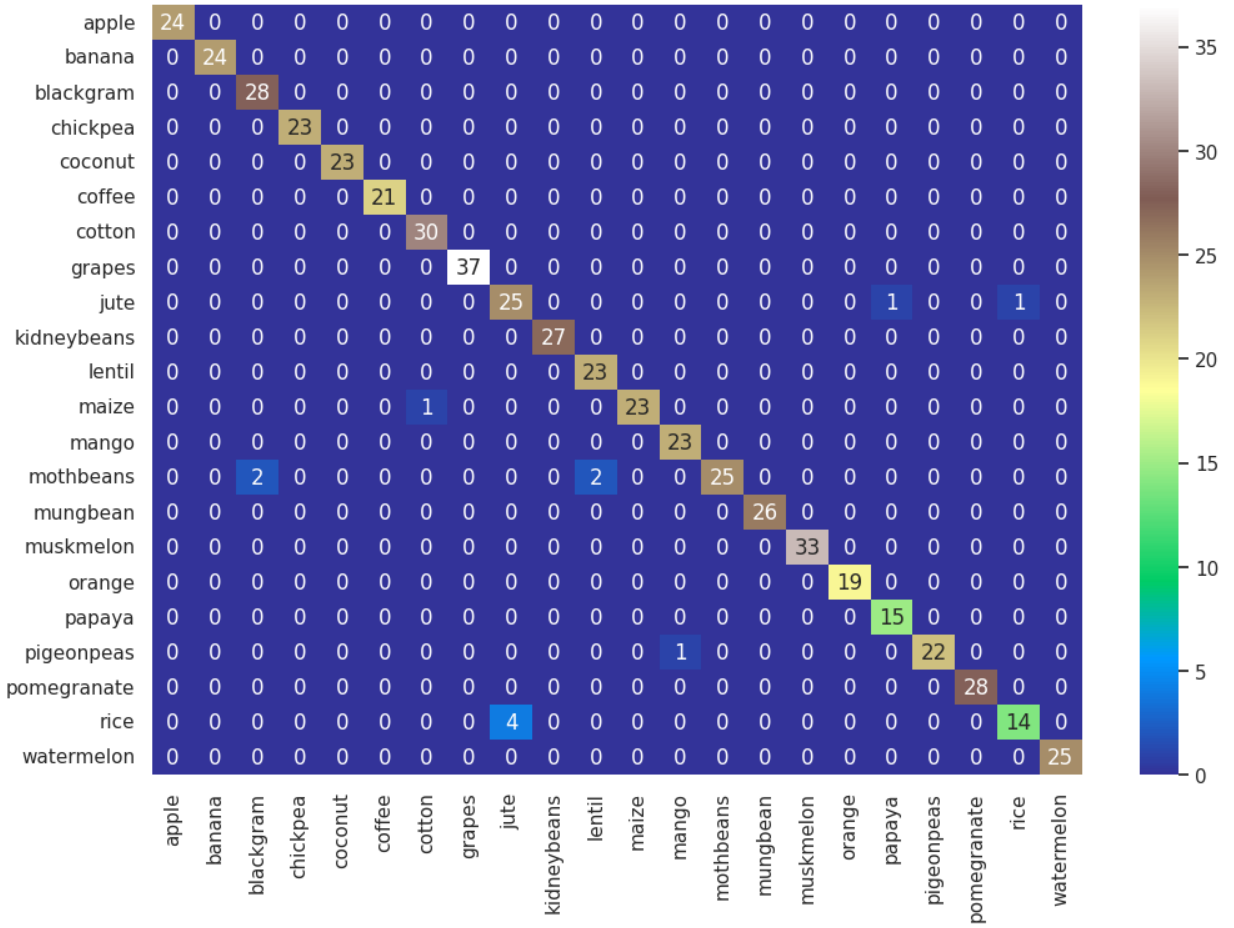

Figure: 22×22 Confusion Matrix – KNN Classifier Results (terrain colormap)

Crop	Correct	Total	Accuracy	Misclassified as
apple	24	24	100.0%	—
banana	24	24	100.0%	—
blackgram	28	28	100.0%	—
grapes	37	37	100.0%	—
rice	14	18	77.8%	4 → jute
jute	25	27	92.6%	1→rice, 1→watermelon
mothbeans	25	27	92.6%	2 → blackgram
maize	23	24	95.8%	1 → cotton
pigeonpeas	22	23	95.7%	1 → maize
OVERALL	430	440	97.82%	10 misclassified total

Step 17 – KNN Prediction on New Sample Data

A KNN model (k=5 explicit) predicts the recommended crop for a new soil/weather sample input.

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
sample = [[20, 42, 43, 28, 70, 6.5, 180]]   # [N, P, K, temp, humidity, ph, rainfall]
```

```
sample_scaled = scaler.transform(sample)
pred_num  = knn.predict(sample_scaled)
pred_crop = le.inverse_transform(pred_num)
print('Predicted Crop:', pred_crop[0])
```

```
Input:  N=20, P=42, K=43, Temperature=28°C, Humidity=70%, pH=6.5, Rainfall=180mm
🌱 Predicted Crop: pigeonpeas
→ For the given soil and climate conditions, pigeonpeas is the recommended crop.
```

5. Combined Summary & Conclusion

Both laboratory practicals together form a comprehensive Data Engineering workflow — from raw file ingestion through to a production-ready machine learning classifier.

#	Lab	Task / Step	Outcome
1	LAB-1	CSV/JSON/HTML/XML/TXT Parsing	All formats read and written successfully
2	LAB-1	Binary File I/O (pickle)	DataFrame serialized and restored without data loss
3	LAB-1	Regex Operations	Pattern search, replace (maize→corn), and split
4	LAB-1	SQL CRUD – SQLite	Full CRUD on crop.db; 2200 rows managed
5	LAB-1	MongoDB – PyMongo	Code demonstrated; server unavailable in Colab
6	LAB-1	NumPy Array Operations	Reshape, slice, index, column-wise mean
7	LAB-1	Pandas Data Wrangling	Indexing, Boolean filter, groupby, inner merge
8	LAB-1	Data Visualization	Histogram & boxplot for rainfall distribution
9	LAB-1	ETL Verification (CSV write/read)	Data integrity confirmed through full pipeline
10	LAB-2	Data Load & Structure Check	2200×8 dataset loaded; shape and dtypes confirmed
11	LAB-2	Statistical Summary	Mean, std, min/max computed for all features
12	LAB-2	Unique Values & Missing Check	137 unique N values; zero missing values
13	LAB-2	Class Distribution	Perfectly balanced: 22 crops × 100 samples
14	LAB-2	Box Plots	Outliers visualised for all 7 features
15	LAB-2	IQR Outlier Analysis	No extreme humidity outliers detected
16	LAB-2	Overall Summary Statistics	Global averages computed for all features
17	LAB-2	Interactive Per-Crop Stats	ipywidgets @interact for dynamic crop stats
18	LAB-2	Distribution Plots	Multimodal and right-skewed distributions found
19	LAB-2	Relational Scatter Plot	Temperature–Humidity–Rainfall relationship plotted
20	LAB-2	Season-wise Classification	Summer / Winter / Rainy crops identified
21	LAB-2	K-Means Clustering	4 natural crop clusters discovered
22	LAB-2	Correlation Heatmap	P–K strong correlation (74%); others weak
23	LAB-2	Encoding, Scaling & 80/20 Split	LabelEncoder + MinMaxScaler + train/test split

#	Lab	Task / Step	Outcome
24	LAB-2	KNN Training & Accuracy	97.82% test accuracy achieved
25	LAB-2	Confusion Matrix	22×22 matrix; most crops classified perfectly
26	LAB-2	New Sample Prediction	Sample [N=20,P=42,...] predicted as 'pigeonpeas'

The KNN classifier achieved 97.82% accuracy (430/440 correct) on the held-out test set, demonstrating that soil nutrient levels and weather parameters are highly predictive of the optimal crop. Minor misclassifications occurred between agronomically similar crops (rice/jute share high rainfall requirements). The dataset's perfect class balance (100 samples per crop) ensured unbiased training. K-Means clustering independently confirmed that crops form four natural groupings aligned with climate and soil profiles — validating the feature space before supervised learning.

6. References

Dataset

[1] Ingle, A. (2020). Crop Recommendation Dataset. Kaggle. Retrieved from https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset

Core Python Libraries

[2] McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 445, 51–56. (pandas library)

[3] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. Nature, 585, 357–362. https://doi.org/10.1038/s41586-020-2649-2

[4] Waskom, M. (2021). seaborn: Statistical data visualization. Journal of Open Source Software, 6(60), 3021. https://doi.org/10.21105/joss.03021

[5] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Machine Learning

[6] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://scikit-learn.org/

[7] Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. IEEE Transactions on Information Theory, 13(1), 21–27. https://doi.org/10.1109/TIT.1967.1053964

[8] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, 1(14), 281–297.

Database & File I/O

[9] Python Software Foundation. (2024). sqlite3 — DB-API 2.0 interface for SQLite databases. Python 3 Documentation. https://docs.python.org/3/library/sqlite3.html

[10] Python Software Foundation. (2024). pickle — Python object serialization. Python 3 Documentation. https://docs.python.org/3/library/pickle.html

[11] Python Software Foundation. (2024). re — Regular expression operations. Python 3 Documentation. https://docs.python.org/3/library/re.html

[12] MongoDB, Inc. (2024). PyMongo documentation. https://pymongo.readthedocs.io/

Visualisation & Interactive Tools

[13] Kluyver, T., Ragan-Kelley, B., Pérez, F., et al. (2016). Jupyter Notebooks — a publishing format for reproducible computational workflows. In Positioning and Power in Academic Publishing (pp. 87–90). IOS Press.

[14] Grout, J., Corlay, S., Ivanov, P., et al. (2021). ipywidgets: Interactive HTML widgets for Jupyter notebooks. GitHub. https://github.com/jupyter-widgets/ipywidgets

Platform

[15] Google LLC. (2017). Google Colaboratory: A free Jupyter notebook environment running in the cloud. https://colab.research.google.com/

[16] Bisong, E. (2019). Google Colaboratory. In Building Machine Learning and Deep Learning Models on Google Cloud Platform (pp. 59–64). Apress, Berkeley, CA.